

TP Grupal: Money Laundering Analysis

Diseño

Facultad de Ingeniería de la Universidad de Buenos Aires

Sistemas Distribuidos

Cátedra Roca



Integrantes:

Alumno	Padrón
Fernandez, Ignacio Agustín	111019
Molina, Taiel Alexis	109458
Serra, Luciano	110694

Propuesta

Vista lógica

DAG

Representación de las transformaciones que presentan los datos en los saltos entre instancias del sistema, así como las dependencias de secuencialidad entre las mismas.

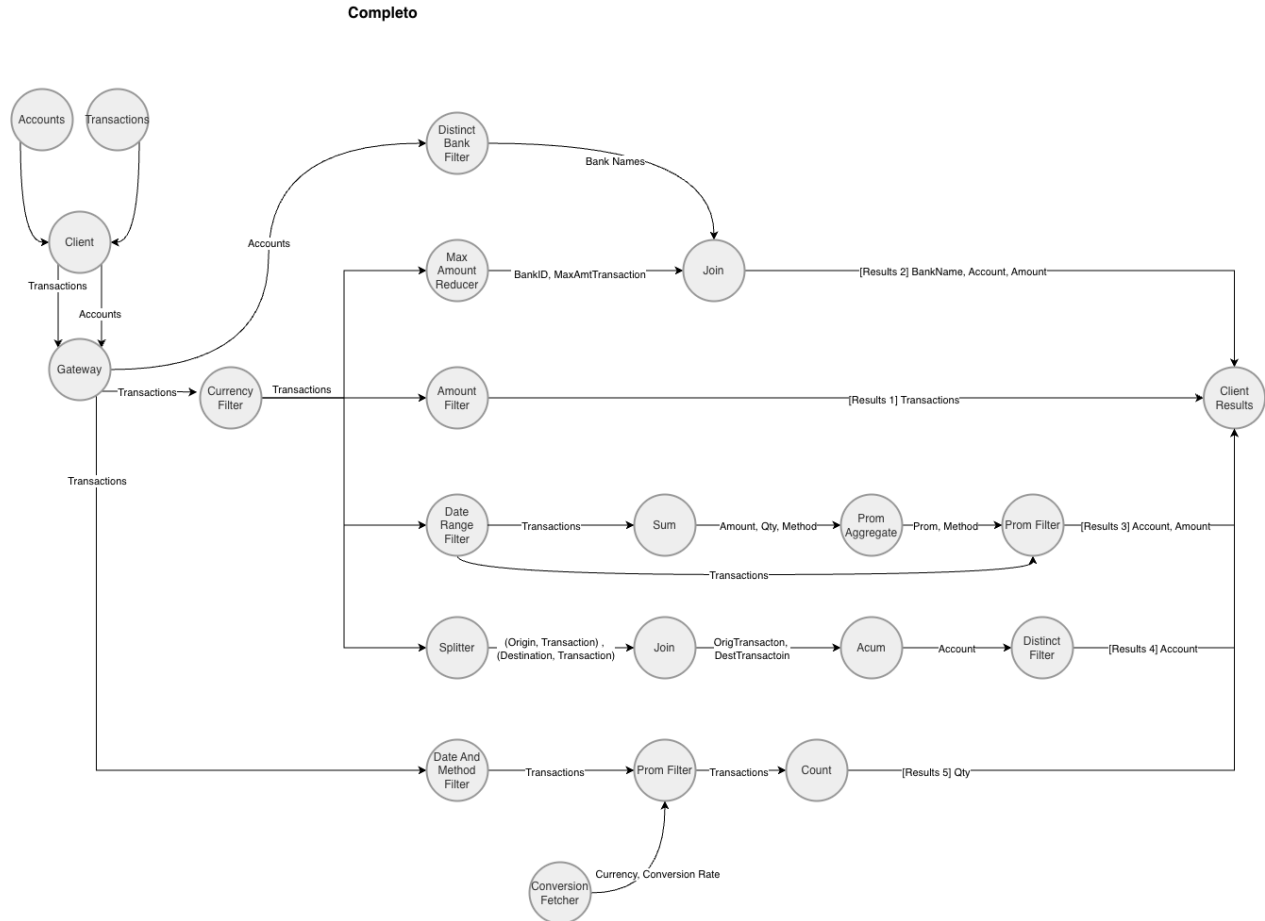


Figure 1: DAG de todo el sistema.

Sumamos detalle "haciendo zoom" en cada una de las queries entendiendo el funcionamiento aislado de cada una.

Para estos añadimos un detalle que muestra la dependencia entre cada instancia. Las relaciones "stream" implican que el nodo origen puede transmitir datos al nodo destino sin necesidad de haber procesado su correspondiente EOF.

1. Cuenta de origen, cuenta de destino y monto para transacciones USD menores a 50.



Figure 2: DAG ejercicio 1.

2. Nombre de banco, cuenta de origen y monto de la max. transacción USD de cada banco.

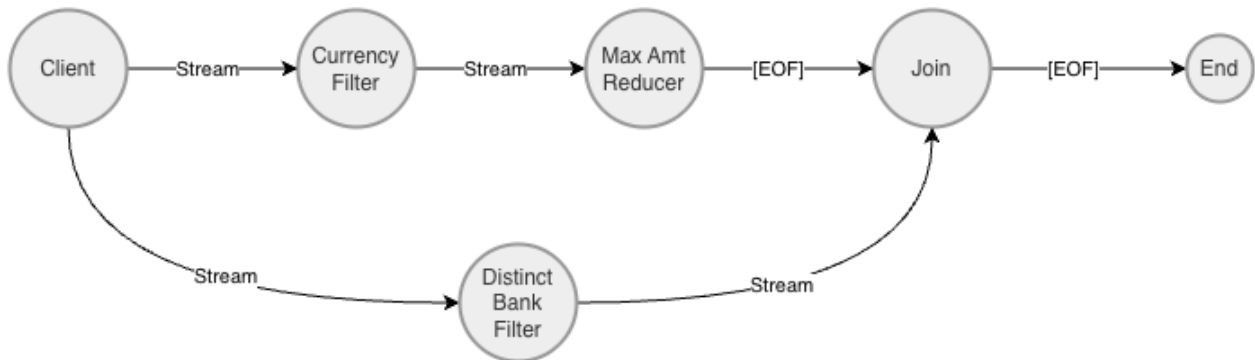


Figure 3: DAG ejercicio 2.

3. Cuenta de origen y monto de transacciones USD en el período [2022-09-06, 2022-09-15] con monto menor a 1 centésimo del promedio encontrado para el mismo formato de pago en el período [2022-09-01, 2022-09-05]

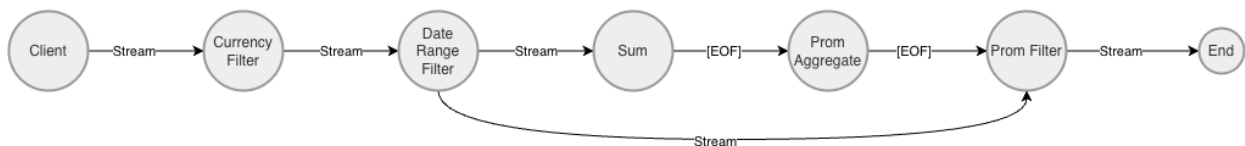


Figure 4: DAG ejercicio 3.

4. Cuentas que cumplan con el patrón scatter-gather con una sola cuenta de separación, para cuentas que hayan realizado y cuya cuenta de origen haya realizado transferencias USD hacia entre 5 cuentas distintas dentro del período [2022-09-01, 2022-09-05]



Figure 5: DAG ejercicio 4.

5. Cantidad de transacciones del período [2022-09-01, 2022-09-05] con formato de pago "Wire" o "ACH" cuyo monto convertido a USD sea menor a 1

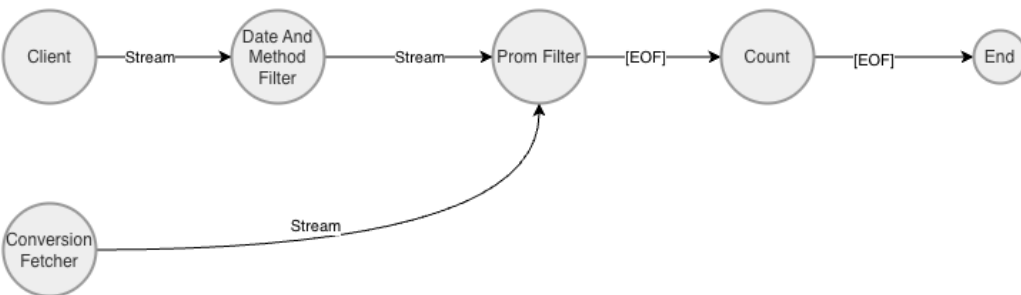


Figure 6: DAG ejercicio 5.

Vista física

Diagrama de robustez

En las primeras cuatro queries dejamos remarcado punteado en verde la parte del procesamiento que es compartido, el cual es el envío de los clientes al gateway de las transferencias y las cuentas, y un nodo Filter del tipo Currency va a pasarle a otro worker solo las transferencias en USD.

Query I

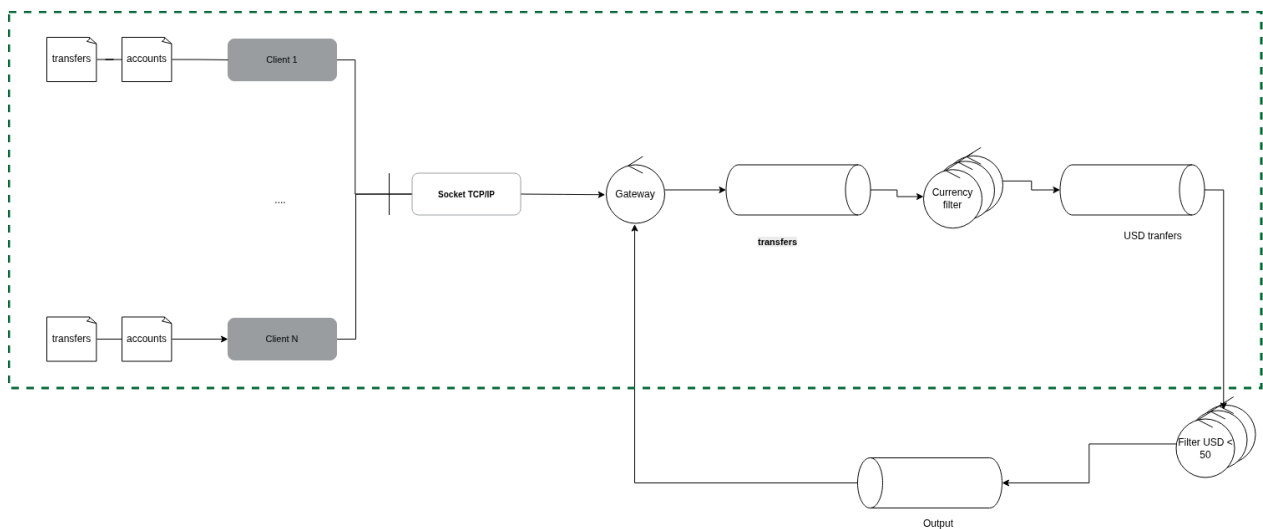


Figure 7: Query I.

Query II

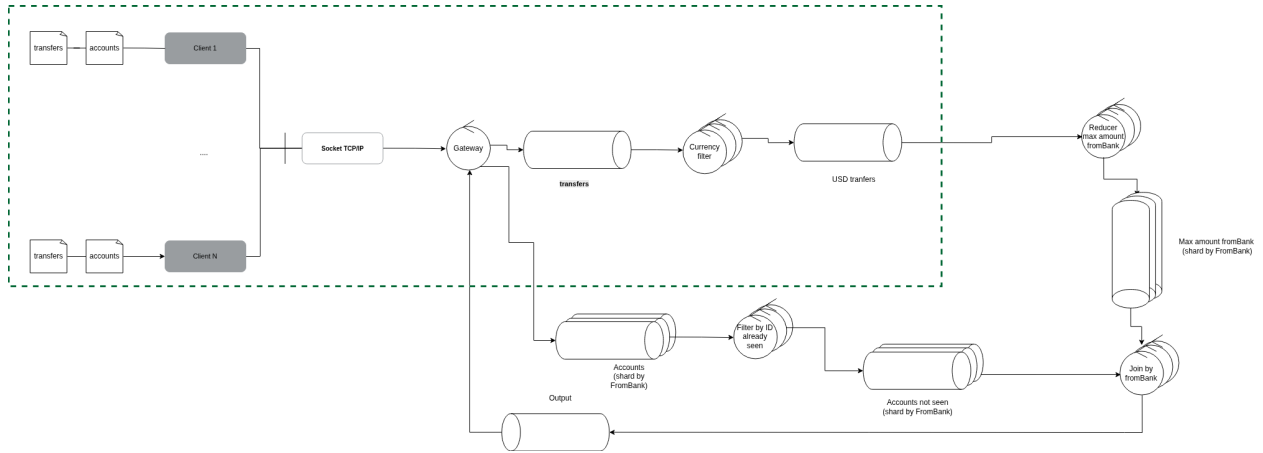


Figure 8: Query II.

Query III

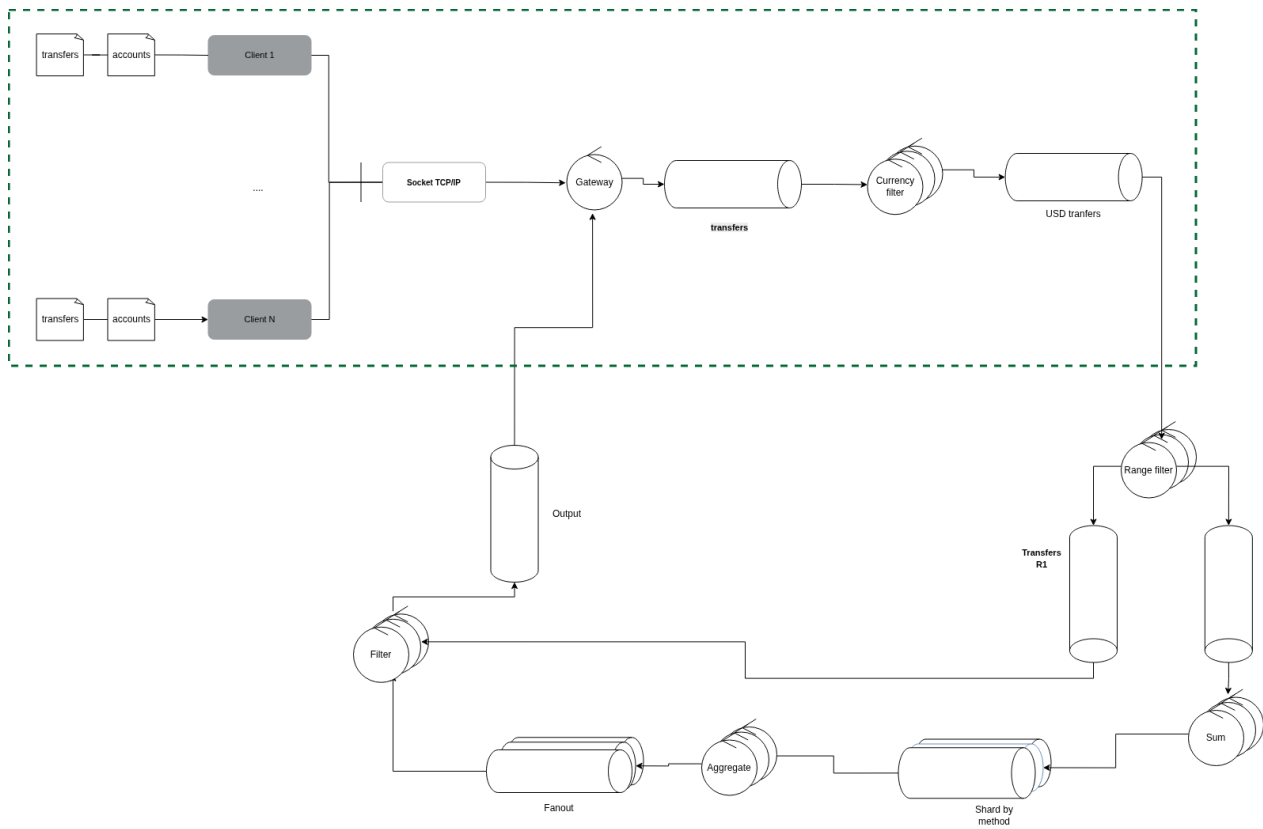


Figure 9: Query III.

Query IV

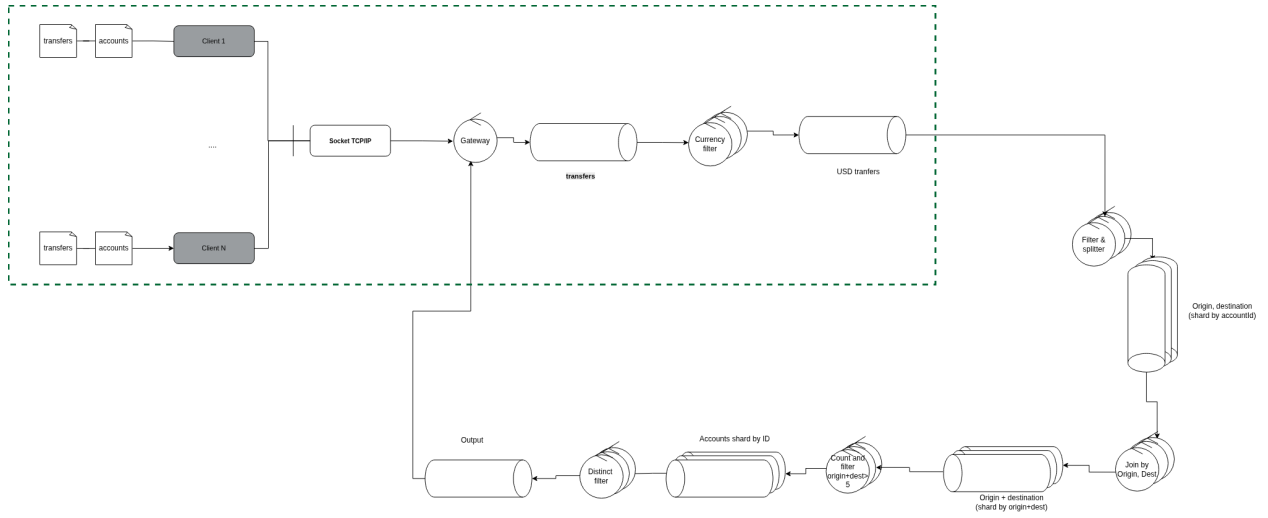


Figure 10: Query IV.

Query V

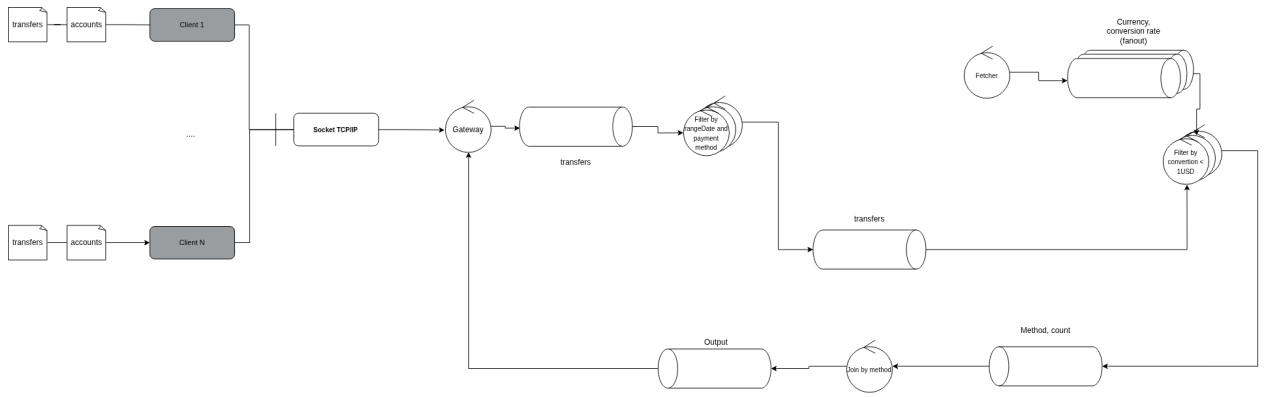


Figure 11: Query V.

Diagrama de despliegue

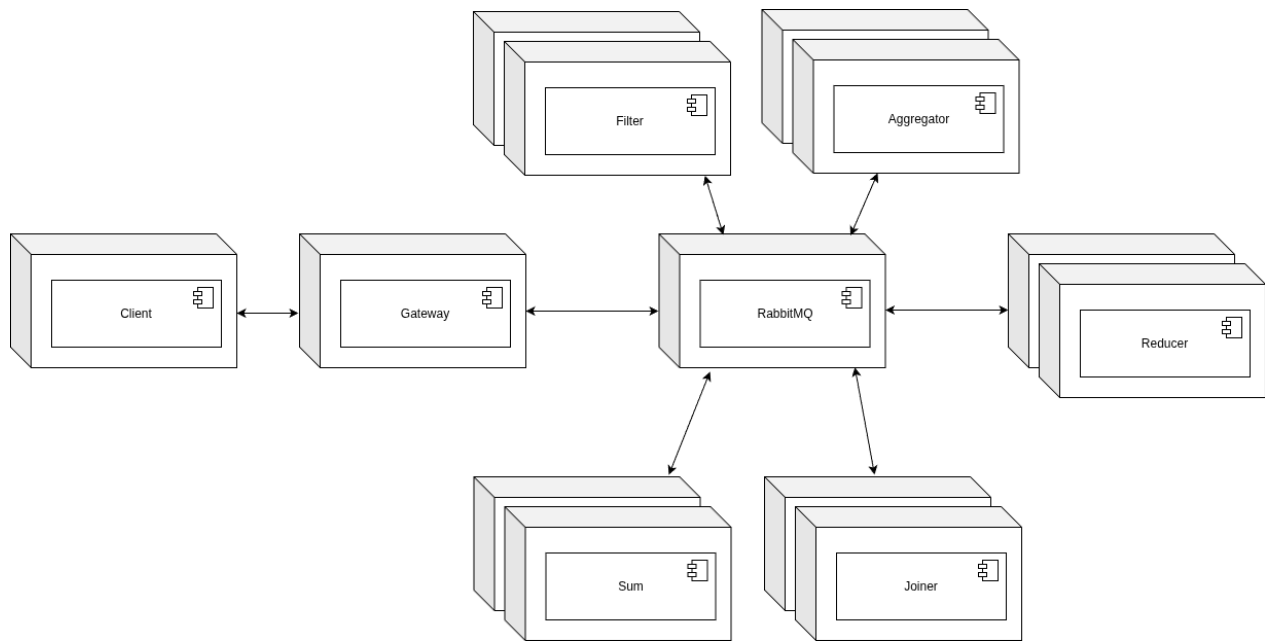


Figure 12: Diagrama de despliegue del sistema.

En este diagrama se puede visualizar cómo se distribuyen los componentes físicos del sistema. Se puede observar que los nodos de procesamiento, como el Filter, Reducer, Joiner, Aggregator y Counter, se representan con múltiples instancias, indicando que pueden escalar horizontalmente agregando más réplicas según la carga. Todos estos nodos se comunican a través de RabbitMQ, que centraliza el flujo de mensajes del sistema. El Client y el Gateway se despliegan como instancias únicas, siendo este último el encargado de recibir los datos del cliente y distribuirlos hacia las colas correspondientes.

Vista de desarrollo

Diagrama de paquetes

Se presenta un paquete **common** compartido por todos los componentes del sistema, el cual contiene los módulos de **Middleware** (wrapper de RabbitMQ), **Protocol** (serialización de mensajes) y **Models** (estructuras de datos compartidas).

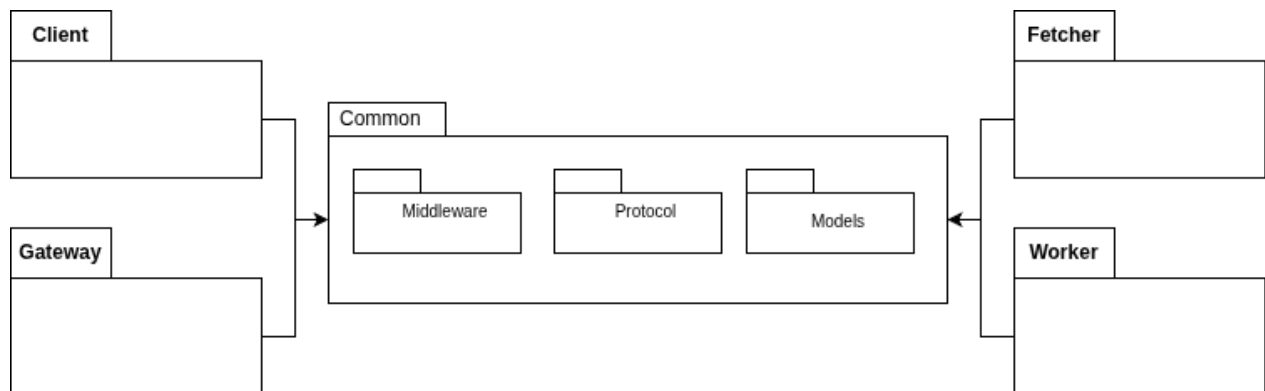
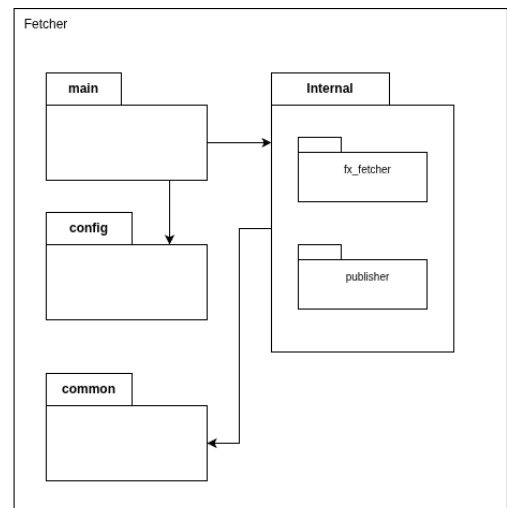
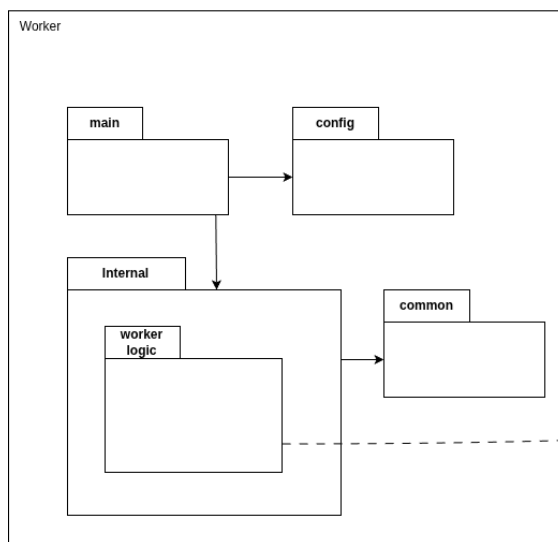
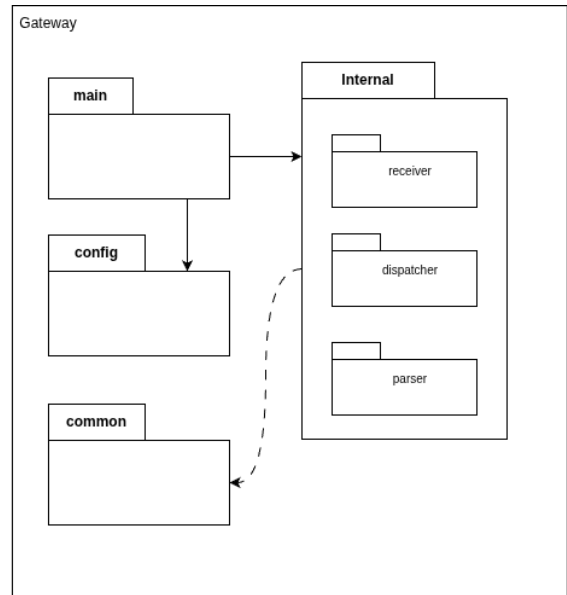
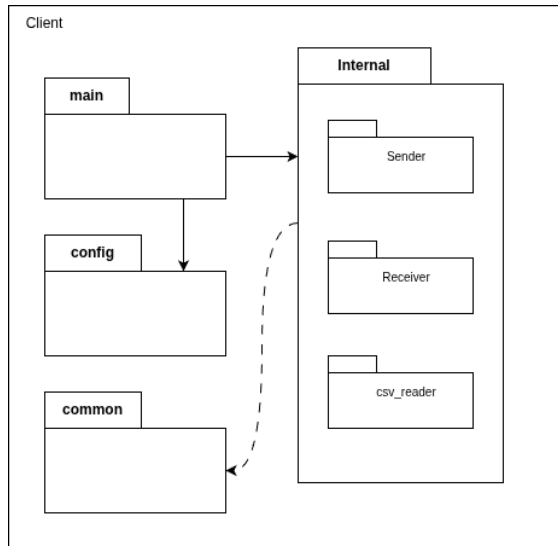


Figure 13: Vista general de dependencias entre paquetes del sistema.

A continuación se detalla la estructura interna de cada paquete. Todos comparten una estructura similar: un módulo **main** como punto de entrada, un módulo **config** para las variables de entorno, y un paquete **internal** con la lógica específica de cada componente. El **Client** contiene los módulos de envío del CSV y recepción del reporte final. El **Gateway** es el punto de entrada del sistema, y se encarga de recibir los datos del cliente y distribuirlos a las colas correspondientes. El **Fetcher** consulta la API externa de cotizaciones y publica los resultados. Finalmente, el **Worker** es un componente genérico que encapsula la lógica de procesamiento, siendo instanciado con distintos comportamientos según el rol que cumpla en el pipeline (filtrado, reducción, join o agregación).



Vista de procesos

Diagrama de actividades

Query I

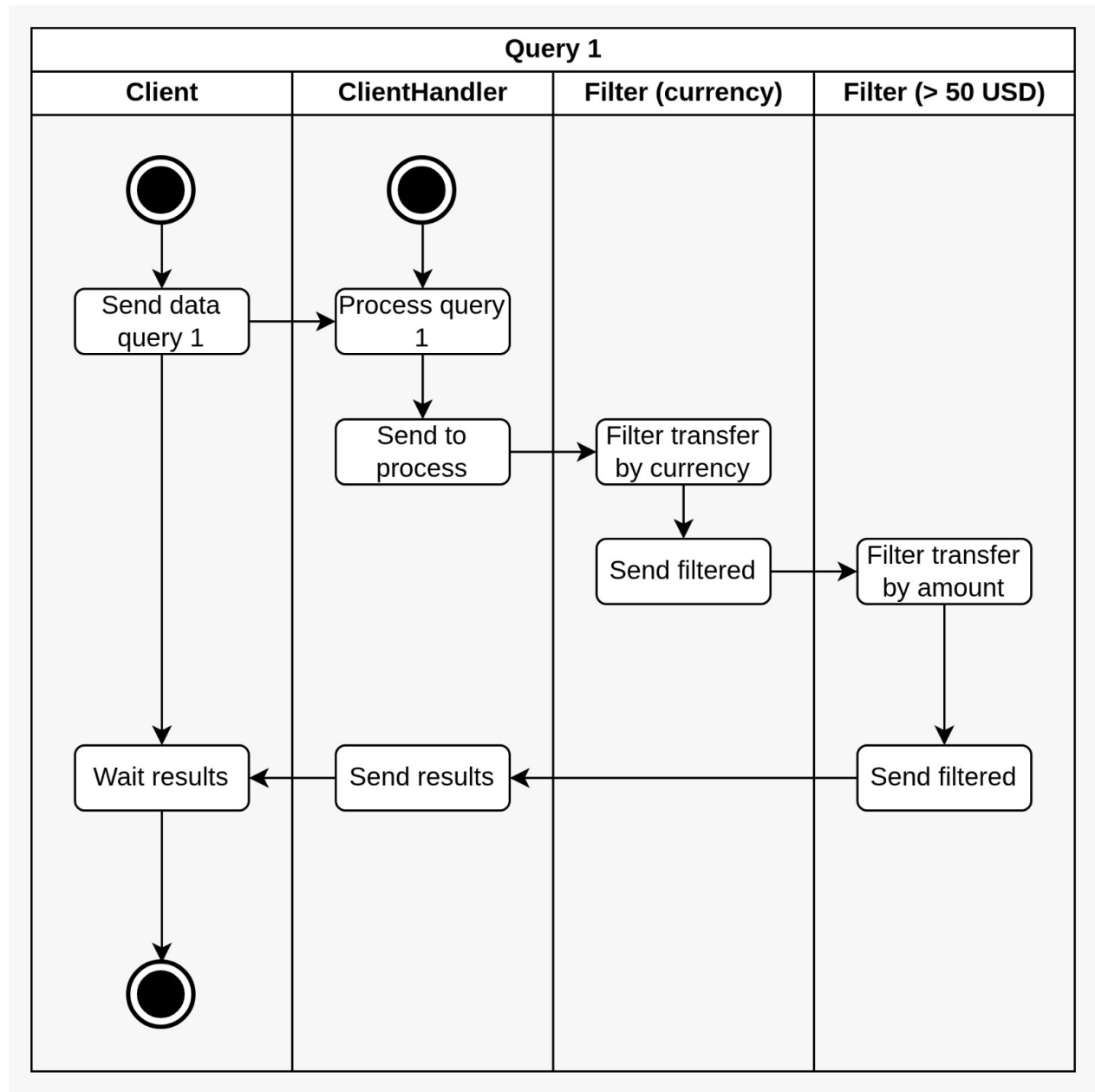


Figure 14: Query I.

Query II

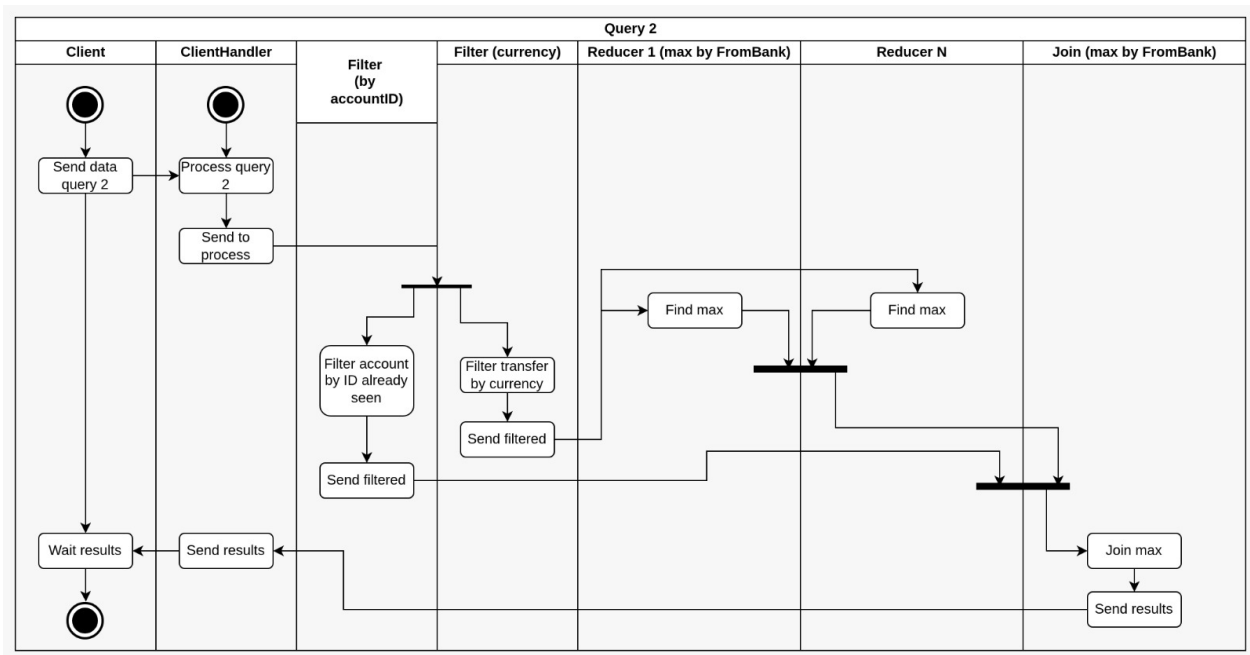


Figure 15: Query II.

Query III

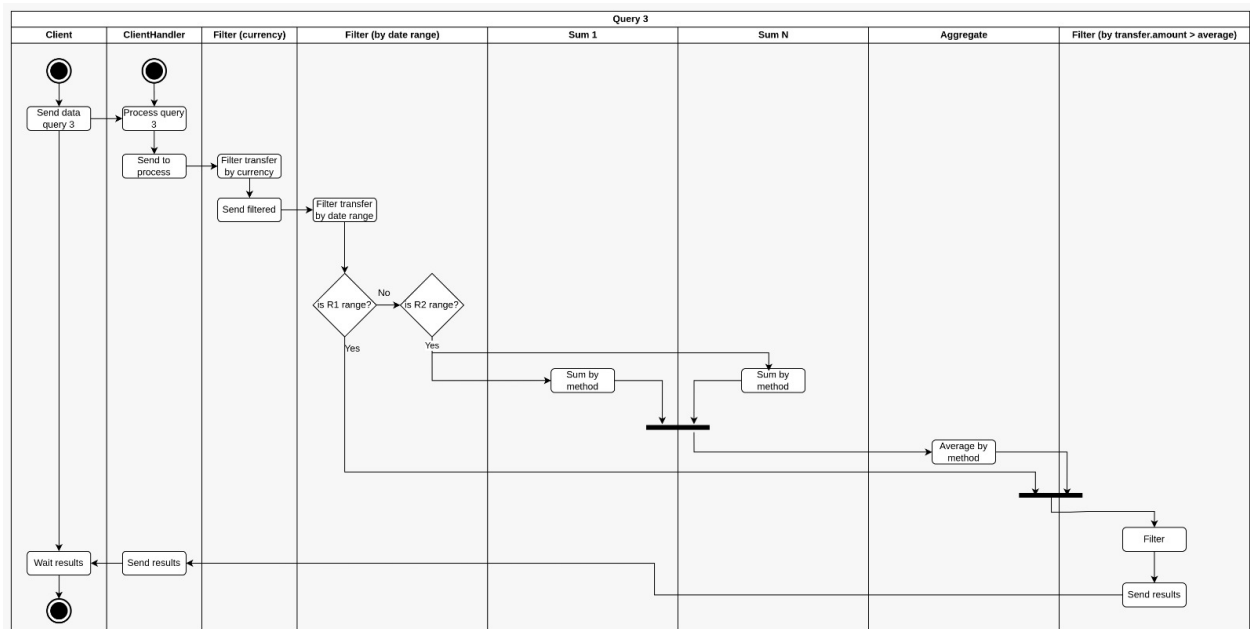


Figure 16: Query III.

Query IV

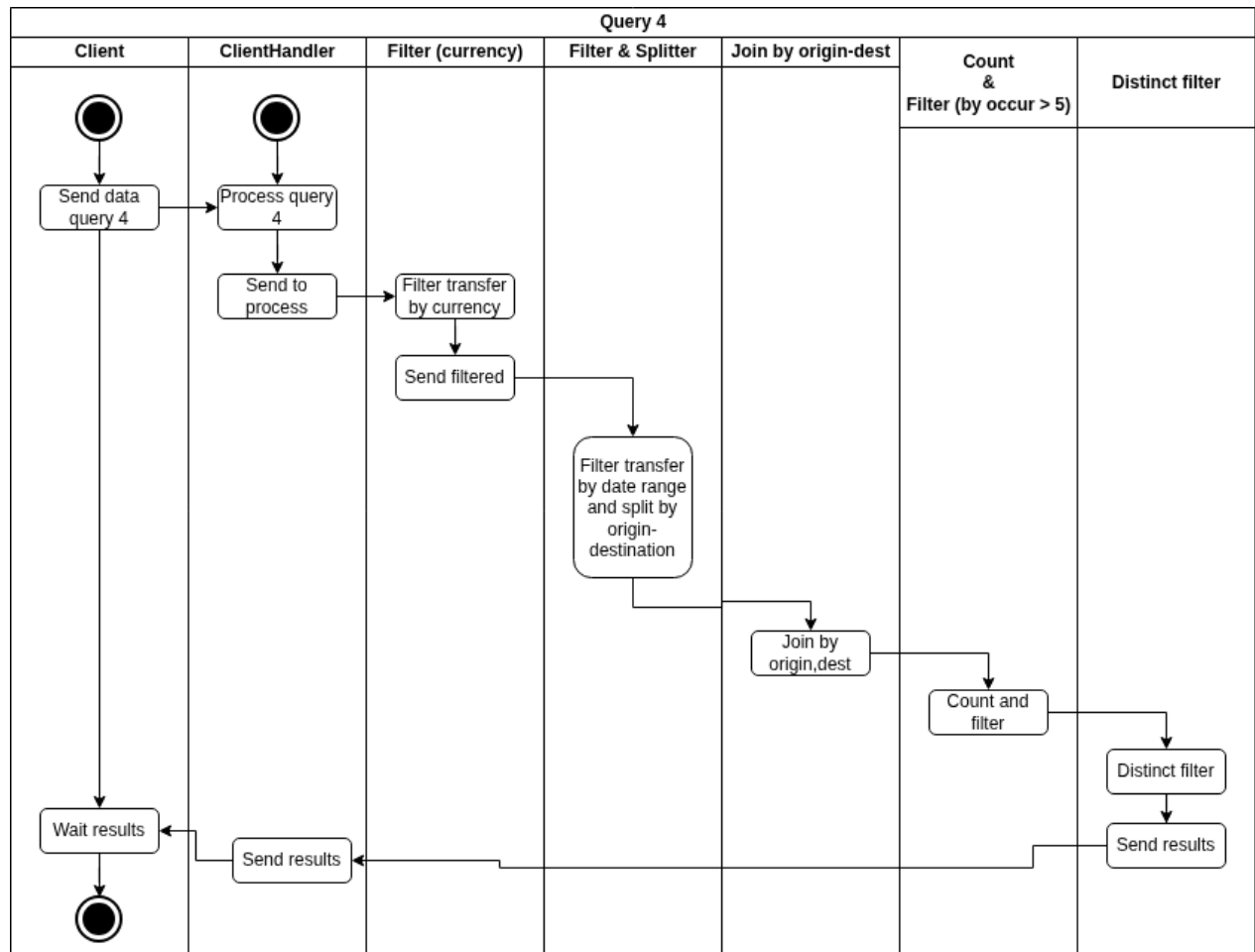


Figure 17: Query IV.

Query V

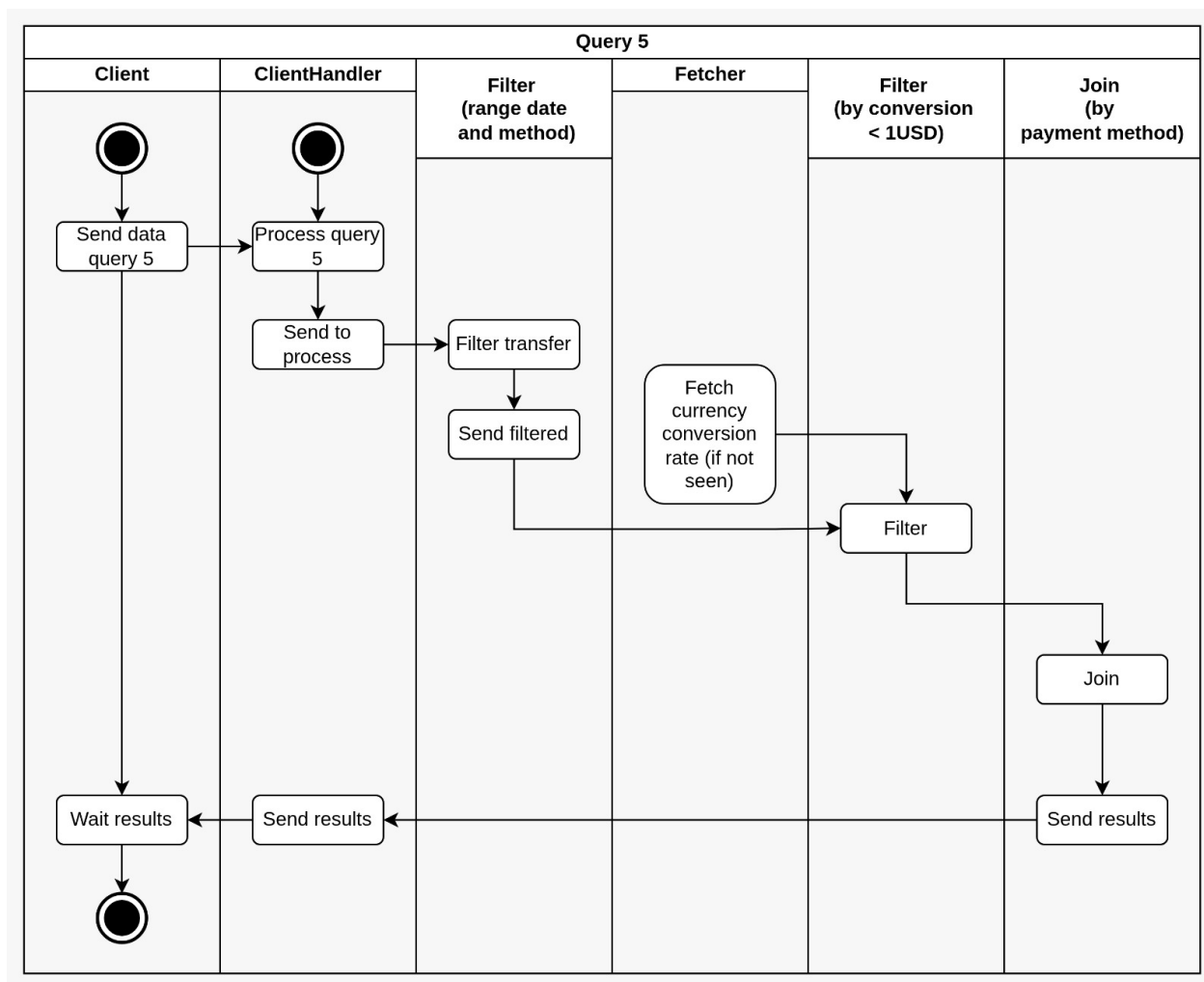


Figure 18: Query V.

Diagrama de secuencia

Con el diagrama de secuencia se busca representar el desacople entre el cliente y el procesamiento del sistema "puertas adentro". El cliente establece conexión con el gateway e inmediatamente inicia la transferencia de los archivos. Priorizamos la carga de las transferencias en primer lugar antes que las cuentas dada la cantidad de operaciones a realizar con estas, además de considerar también el volumen (y de esta forma mantener menos ocioso al sistema). El cliente envía batches de información al gateway el cual los descompone en mensajes a encolar en el exchange/queue correspondiente según la naturaleza del dato.

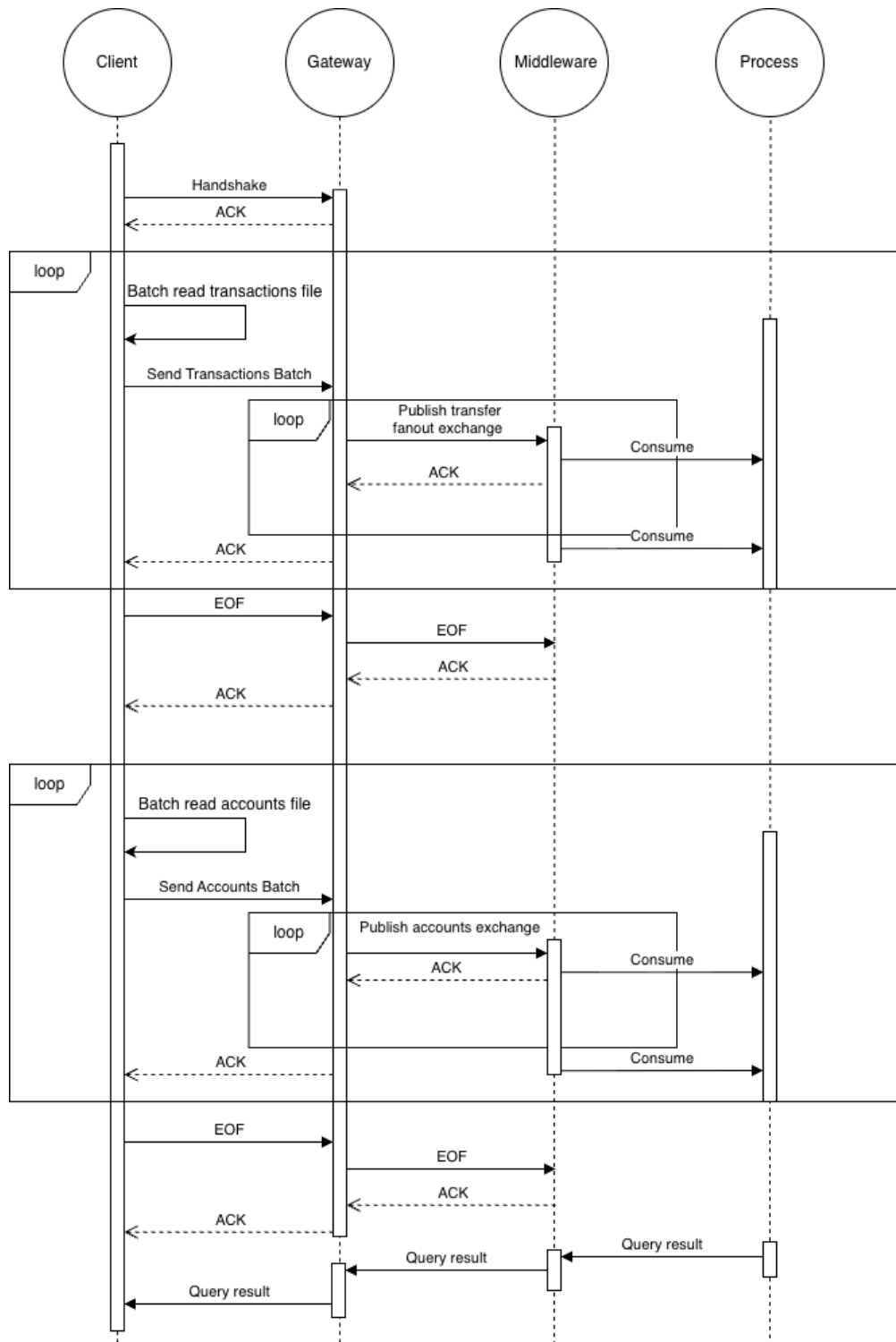


Figure 19: Diagrama de secuencia.

Tareas por hacer

Tras realizar los diagramas, concluimos que la mejor forma de repartir la carga de trabajo consiste en tomar cada nodo como una tarea independiente. A estas se suman la adaptación del middleware implementado en

TPs previos para que se ajuste a nuestro sistema, junto con el armado del cliente y del gateway. Para validar el funcionamiento, también es necesario desarrollar una serie de tests que cubren el happy path, así como un script generador de docker-compose que agilice las tareas. Por último, es requerido mantener actualizado el informe.

Tarea	Responsable
Client + Protocol	Fernandez
Gateway	Serra
Middleware(s)	Serra
Algoritmo sync de EOF	Molina
Currency filter	Molina
Q1 - Filter amount	Molina
Q2 - Reduce	Molina
Q2 - Join	Serra
Q2 - Filter	Molina
Q3 - Range filter	Molina
Q3 - Sum	Serra
Q3 - Aggregate	Fernandez
Q3 - Avg filter	Molina
Q4 - Filter and splitter	Fernandez
Q4 - Join	Serra
Q4 - Acum	Serra
Q4 - Distinct filter	Molina
Q5 - Filter	Molina
Q5 - Fetcher	Fernandez
Q5 - Filter Amount	Molina
Q5 - Join	Serra
Tests Happy Path	Fernandez
Docker compose	Fernandez